EXPERIMENT NO.01

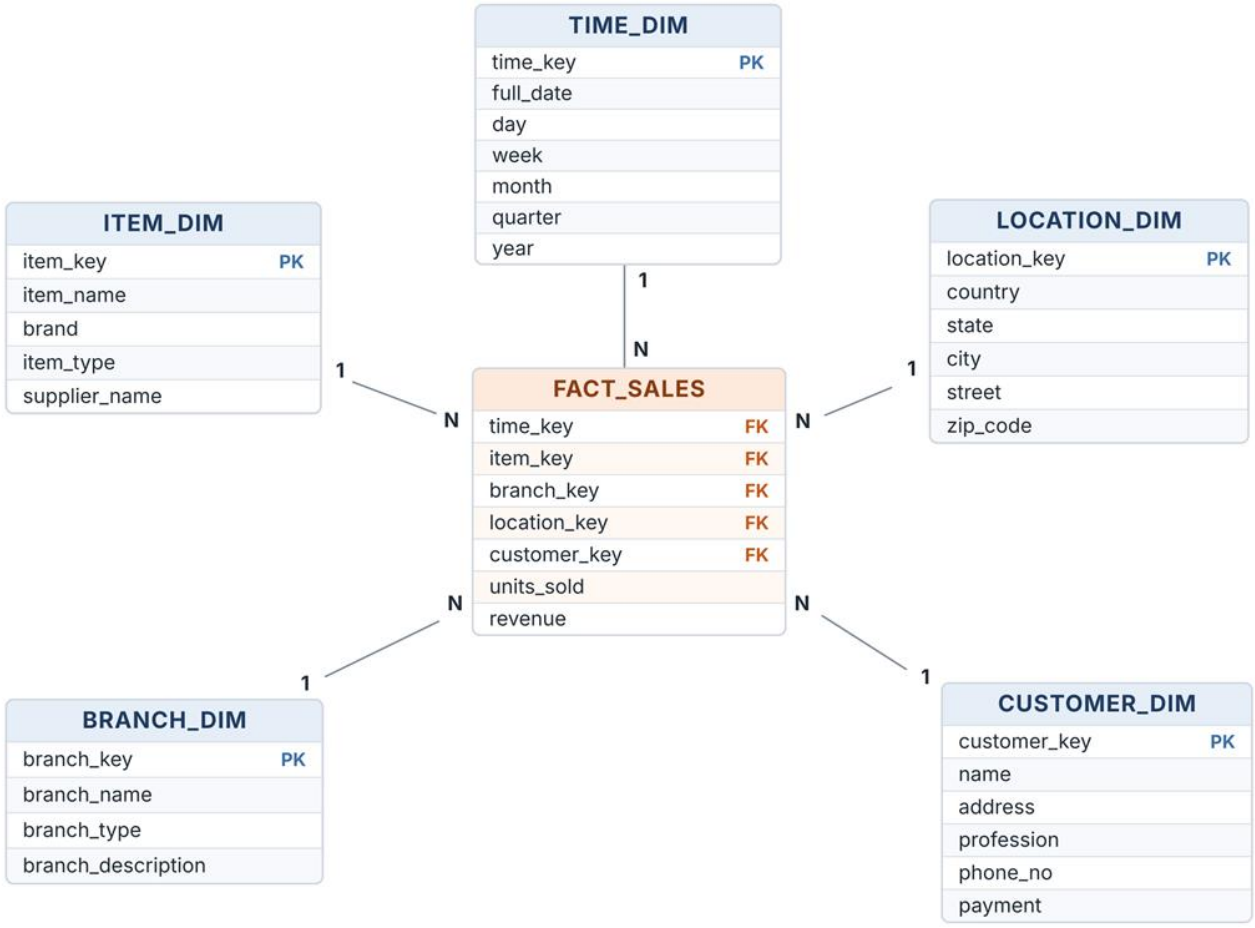


Fig. 1: Star Schema

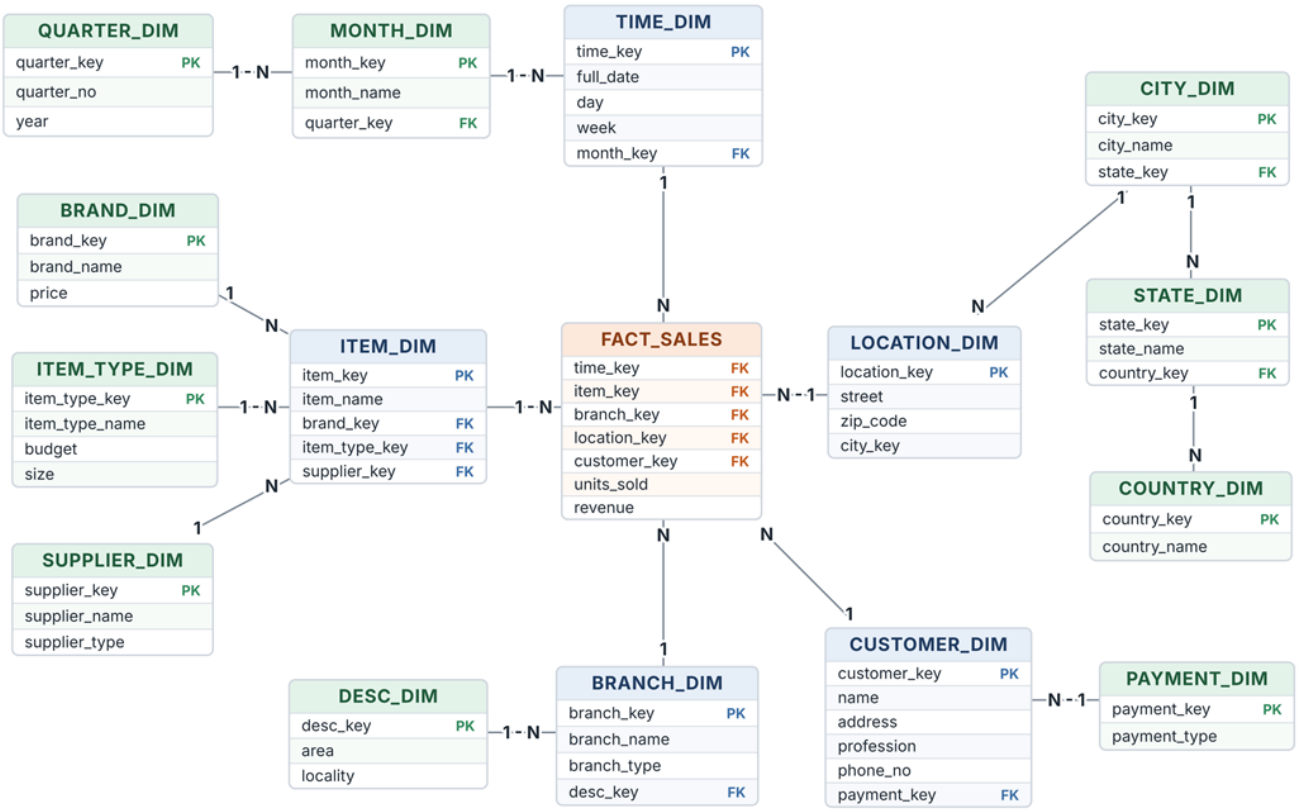


Fig. 2: Snowflake Schema

EXPERIMENT NO.02

```
CREATE DATABASE RETAIL_SALES;
USE RETAIL_SALES;

CREATE TABLE PRODUCT (
  PRODUCT_ID INT PRIMARY KEY,
  PRODUCT_NAME VARCHAR(60),
  CATEGORY VARCHAR(60),
  BRAND VARCHAR(60),
  PRICE INT
);

CREATE TABLE TIME (
  TIME_ID INT PRIMARY KEY,
  FULL_DATE DATE,
  MONTH_NAME VARCHAR(20),
  QUARTER_NO INT,
  YEAR_NO INT
);

CREATE TABLE LOCATION (
  LOCATION_ID INT PRIMARY KEY,
  STREET VARCHAR(60),
  CITY VARCHAR(60),
  STATE VARCHAR(60),
  COUNTRY VARCHAR(60)
);

CREATE TABLE BRANCH (
  BRANCH_ID INT PRIMARY KEY,
  BRANCH_NAME VARCHAR(60)
);

CREATE TABLE SALES (
  PRODUCT_ID INT,
  TIME_ID INT,
  LOCATION_ID INT,
  BRANCH_ID INT,
  UNITS_SOLD INT,
  TOTAL_SALES INT,
  FOREIGN KEY (PRODUCT_ID) REFERENCES PRODUCT(PRODUCT_ID),
  FOREIGN KEY (TIME_ID) REFERENCES TIME(TIME_ID),
  FOREIGN KEY (LOCATION_ID) REFERENCES LOCATION(LOCATION_ID),
  FOREIGN KEY (BRANCH_ID) REFERENCES BRANCH(BRANCH_ID)
);

INSERT INTO PRODUCT VALUES
(1,'RICE','GROCERY','DAWAT',140),
(4,'JACKET','CLOTHING','MAX',700);

INSERT INTO TIME VALUES
(101,'2021-01-07','JANUARY',1,2021),
(102,'2021-02-24','FEBRUARY',1,2021);
```

```
INSERT INTO LOCATION VALUES
(201,'ML ROAD','MUMBAI','MAHARASHTRA','INDIA'),
(202,'AL ROAD','MUMBAI','MAHARASHTRA','INDIA');

INSERT INTO BRANCH VALUES
(301,'ML ROAD BRANCH'),
(302,'AL ROAD BRANCH');

INSERT INTO SALES VALUES
(1,101,201,301,400,56000),
(4,102,202,302,500,350000);

SELECT PRODUCT_NAME, MONTH_NAME, CITY, BRANCH_NAME, UNITS_SOLD, TOTAL_SALES
FROM SALES
JOIN PRODUCT USING (PRODUCT_ID)
JOIN TIME USING (TIME_ID)
JOIN LOCATION USING (LOCATION_ID)
JOIN BRANCH USING (BRANCH_ID);
```

EXPERIMENT NO.03

```
CREATE DATABASE SALES_DW;
USE SALES_DW;

CREATE TABLE TIME (
TIMEID INT PRIMARY KEY,
DAYDATE DATE,
MONTH VARCHAR(20),
QUARTER VARCHAR(5),
YEAR INT
);

CREATE TABLE PRODUCT (
IDPRODUCT INT PRIMARY KEY,
PRODUCTNAME VARCHAR(50),
PRODUCTCATEGORY VARCHAR(50),
PRODUCTPRICE INT
);

CREATE TABLE FACT_SALE (
TIMEID INT,
IDPRODUCT INT,
NUMBER_OF_UNIT_SOLD INT,
FOREIGN KEY (TIMEID) REFERENCES TIME(TIMEID),
FOREIGN KEY (IDPRODUCT) REFERENCES PRODUCT(IDPRODUCT)
);

INSERT INTO TIME VALUES
(1,'2021-01-17','January','Q1',2021),
(2,'2021-02-24','February','Q1',2021),
(3,'2021-05-06','May','Q2',2021);

INSERT INTO PRODUCT VALUES
(1,'Kurta','Cloth',500),
(2,'Headphones','Electronics',1500);

INSERT INTO FACT_SALE VALUES
(1,2,400),
(2,2,300),
(3,1,500);
```

```
SELECT NUMBER_OF_UNIT_SOLD, MONTH
FROM FACT_SALE
JOIN TIME USING (TIMEID)
WHERE QUARTER = 'Q1';

SELECT PRODUCTNAME, NUMBER_OF_UNIT_SOLD
FROM FACT_SALE
JOIN PRODUCT USING (IDPRODUCT)
WHERE PRODUCTCATEGORY = 'Cloth';

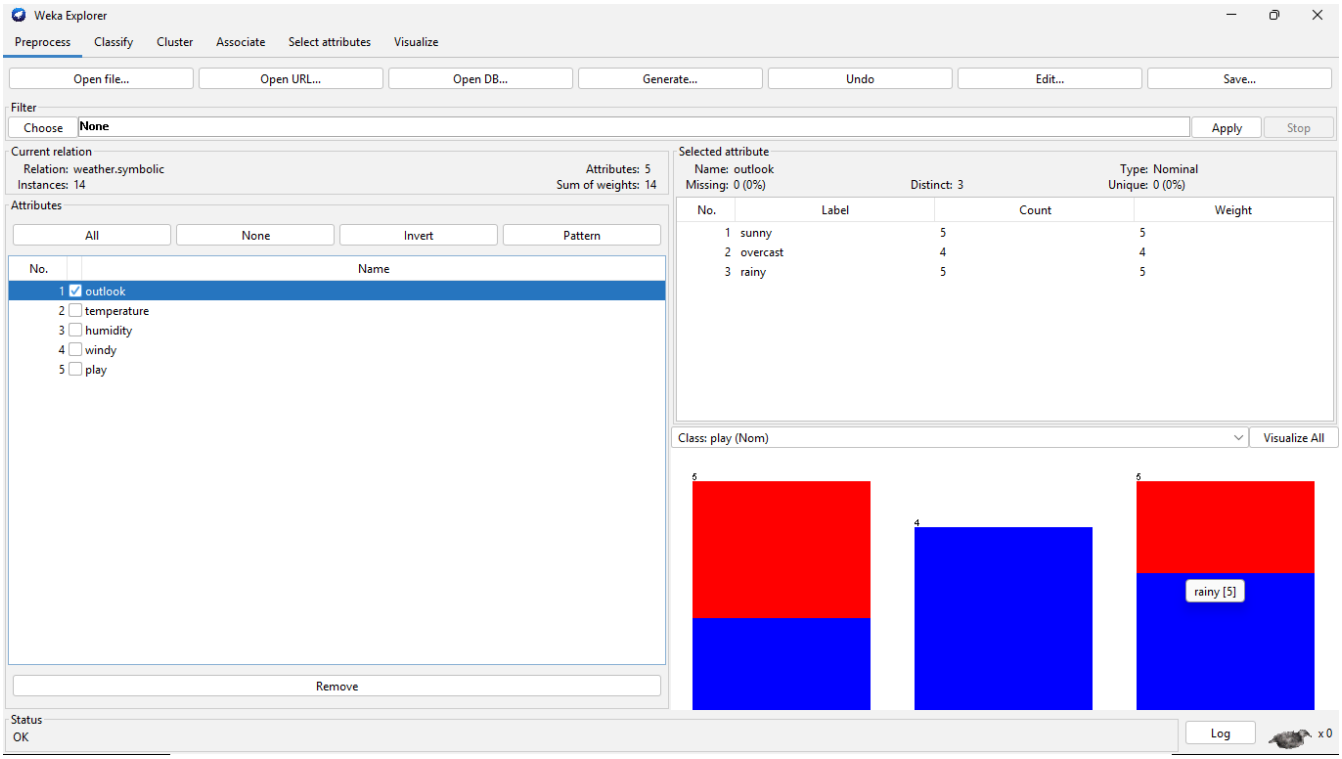
SELECT YEAR, PRODUCTNAME, SUM(PRODUCTPRICE)
FROM FACT_SALE
JOIN PRODUCT USING (IDPRODUCT)
JOIN TIME USING (TIMEID)
WHERE PRODUCTCATEGORY = 'Cloth'
GROUP BY YEAR, PRODUCTNAME;

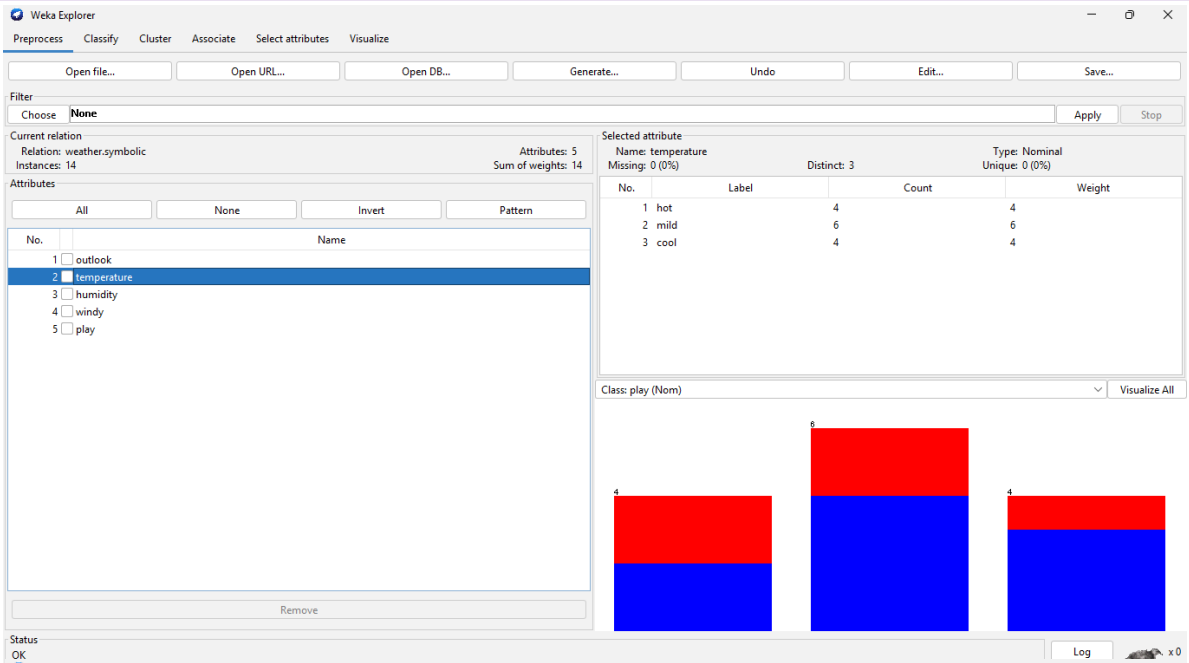
SELECT MONTH, PRODUCTNAME, SUM(PRODUCTPRICE)
FROM FACT_SALE
JOIN PRODUCT USING (IDPRODUCT)
JOIN TIME USING (TIMEID)
WHERE PRODUCTNAME = 'Kurta'
GROUP BY MONTH, PRODUCTNAME;

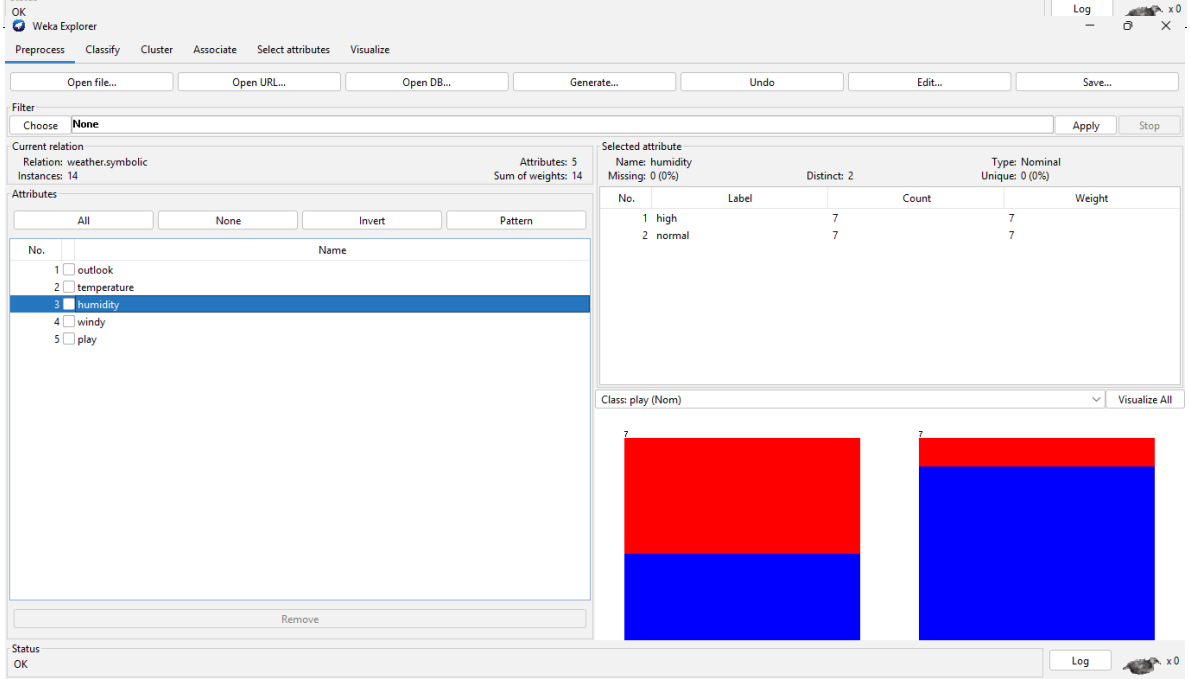
SELECT YEAR,
MAX(CASE WHEN MONTH = 'January' THEN DAYDATE END) AS JANUARY,
MAX(CASE WHEN MONTH = 'February' THEN DAYDATE END) AS FEBRUARY,
MAX(CASE WHEN MONTH = 'May' THEN DAYDATE END) AS MAY
FROM TIME
GROUP BY YEAR;
```

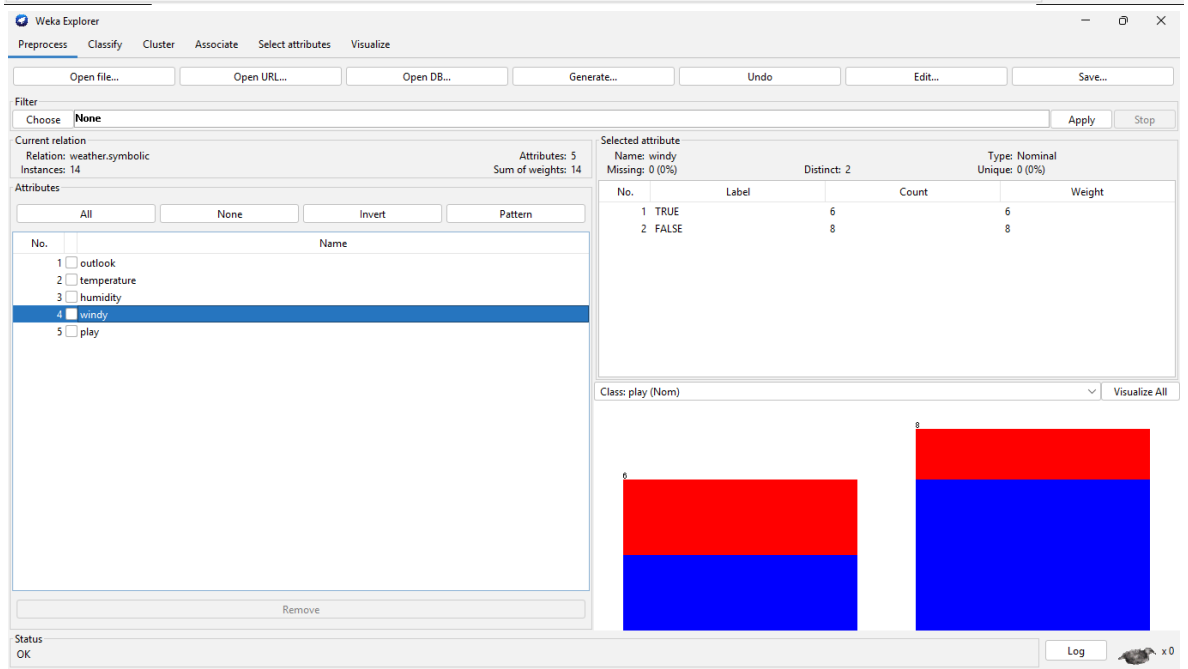
EXPERIMENT NO.04

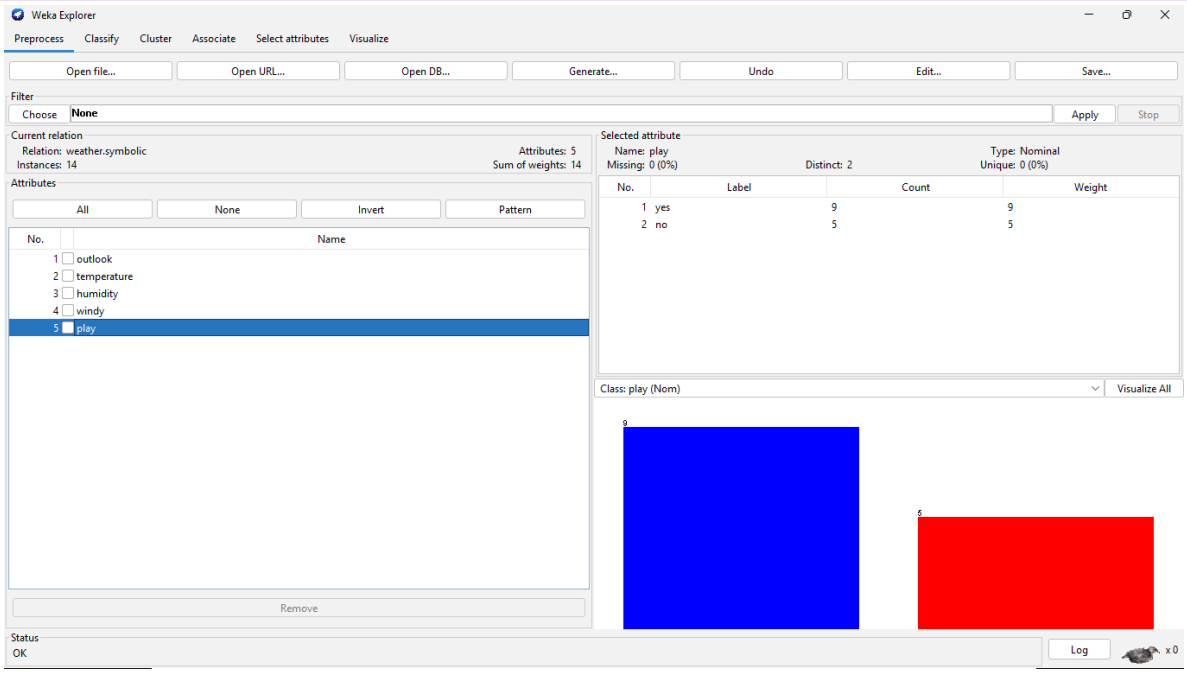
Weather Data Visualization:

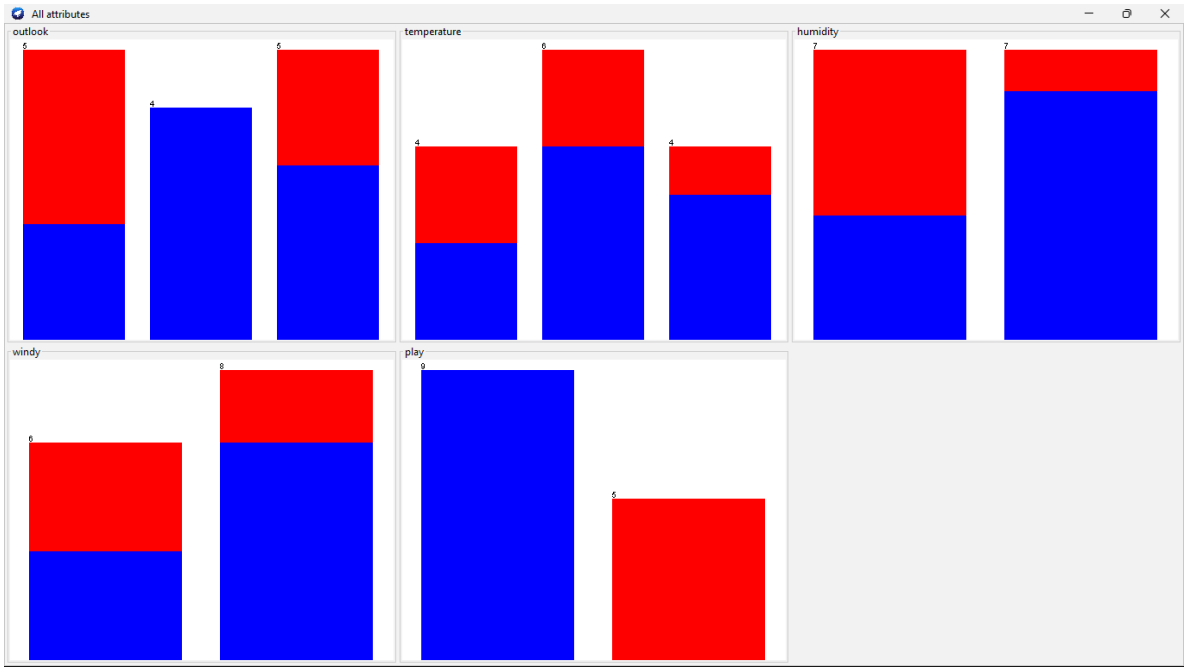



Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Open file...
Open URL...
Open DB...
Generate...
Undo
Edit...
Save...
Filter
Choose
None
Apply
Stop
Current relation
Relation: weather.symbolic
Instances: 14
Attributes: 5
Sum of weights: 14
Attributes
All
None
Invert
Pattern
No.
Name
1 outlook
2 temperature
3 humidity
4 windy
5 play
Remove
Selected attribute
Name: temperature
Missing: 0 (0%)
Distinct: 3
Type: Nominal
Unique: 0 (0%)
No.
Label
Count
Weight
1 hot 4 4
2 mild 6 6
3 cool 4 4
Class: play (Nom)
Visualize All
Status
OK
Log
x 0

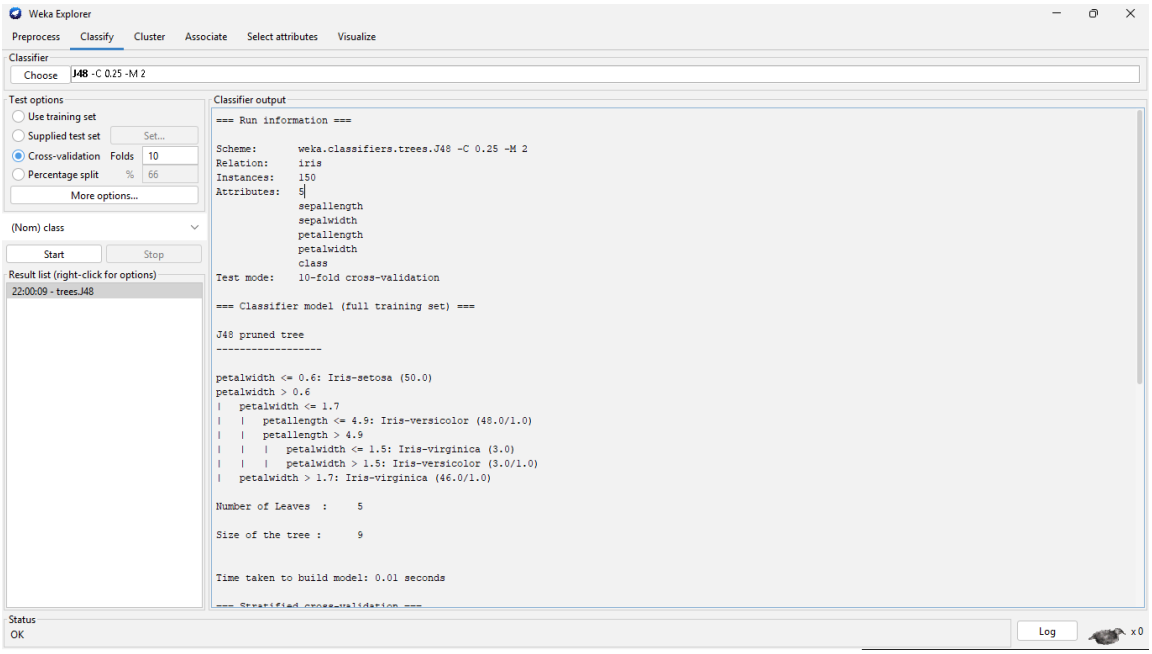

Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Open file...
Open URL...
Open DB...
Generate...
Undo
Edit...
Save...
Filter
Choose
None
Apply
Stop
Current relation
Relation: weather.symbolic
Instances: 14
Attributes: 5
Sum of weights: 14
Attributes
All
None
Invert
Pattern
No.
Name
1 outlook
2 temperature
3 humidity
4 windy
5 play
Remove
Selected attribute
Name: humidity
Missing: 0 (0%)
Distinct: 2
Type: Nominal
Unique: 0 (0%)
No.
Label
Count
Weight
1 high 7 7
2 normal 7 7
Class: play (Nom)
Visualize All
Status
OK
Log
x 0

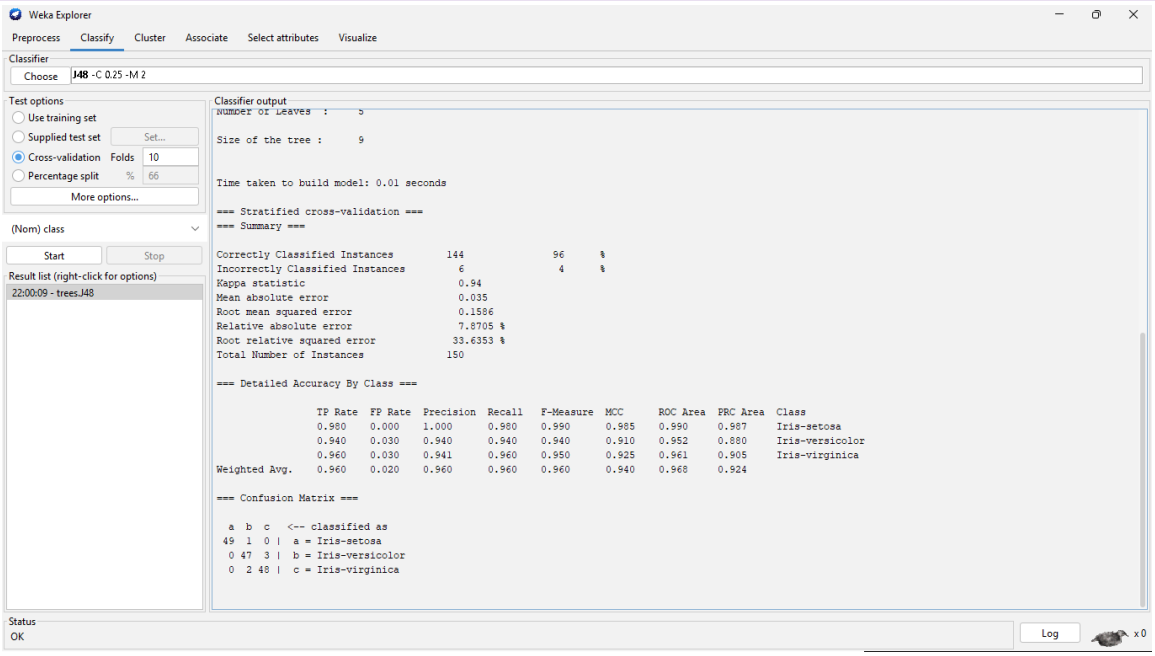

Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Open file...
Open URL...
Open DB...
Generate...
Undo
Edit...
Save...
Filter
Choose
None
Apply
Stop
Current relation
Relation: weather.symbolic
Instances: 14
Attributes: 5
Sum of weights: 14
Attributes
All
None
Invert
Pattern
No.
Name
1 outlook
2 temperature
3 humidity
4 windy
5 play
Remove
Selected attribute
Name: windy
Missing: 0 (0%)
Distinct: 2
Type: Nominal
Unique: 0 (0%)
No.
Label
Count
Weight
1 TRUE 6 6
2 FALSE 8 8
Class: play (Nom)
Visualize All
Status
OK
Log
x 0



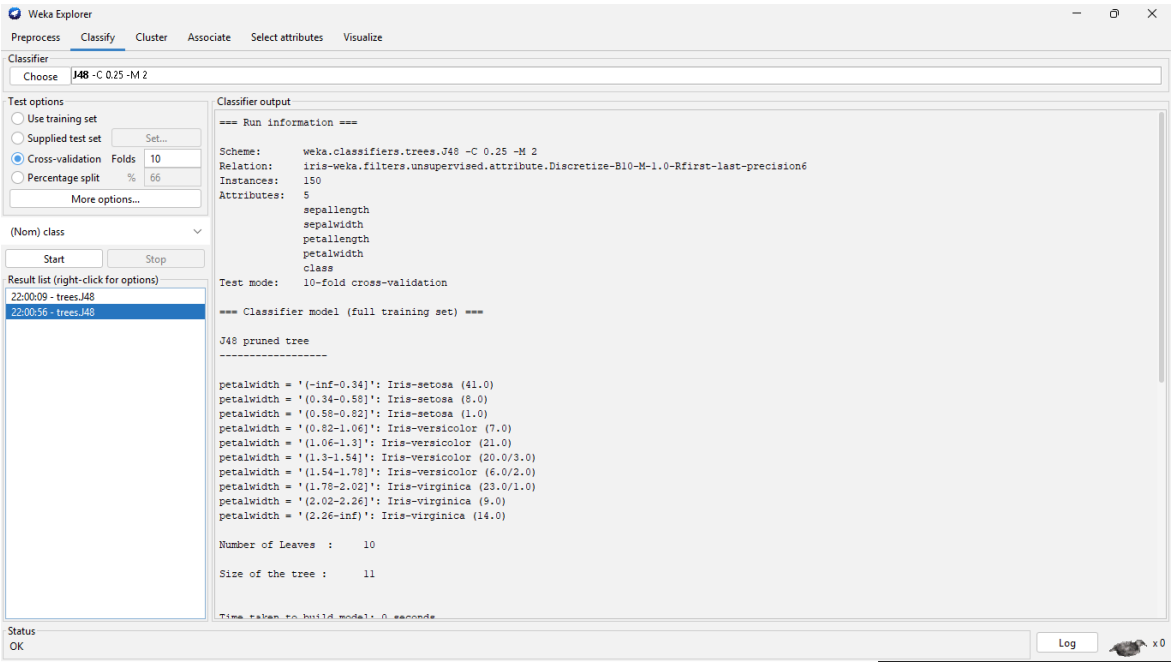


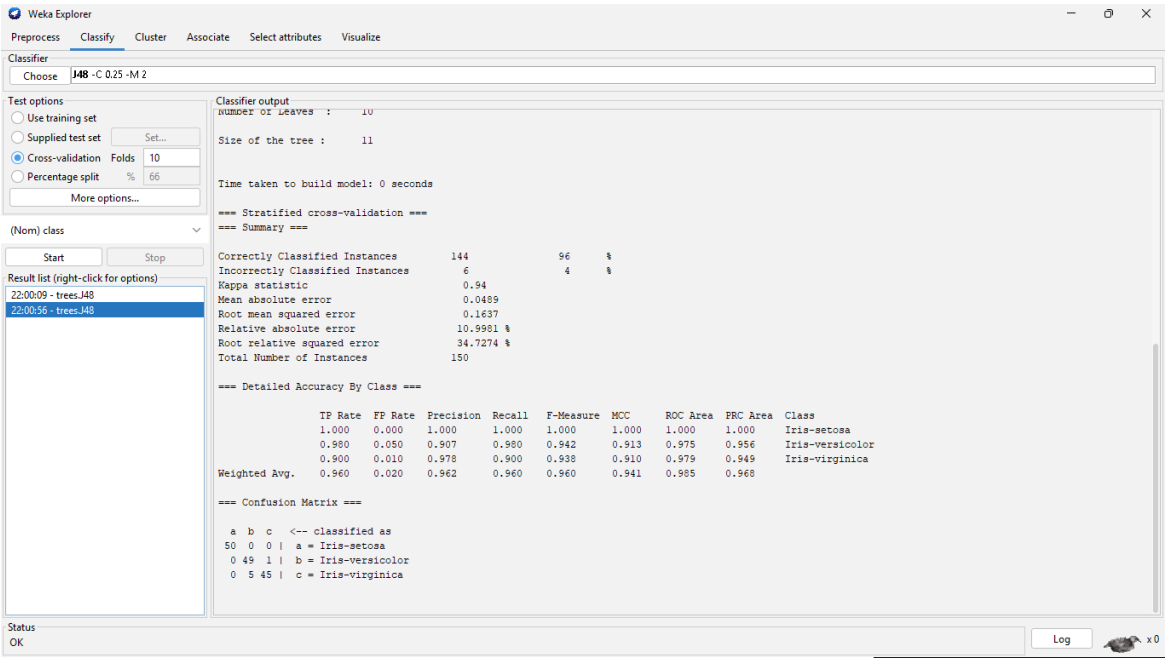
J48 Tree Classification on Iris Dataset before Discretization





J48 Tree Classification on Iris Dataset after Discretization





EXPERIMENT NO.05

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

# Load Iris dataset
data = load_iris()
X = data.data
y = data.target

# Create DataFrame
df = pd.DataFrame(X, columns=data.feature_names)
df["class"] = y
print("First five rows:")
print(df.head())

# Convert class numbers to names
df["class"] = df["class"].map({0: "setosa", 1: "versicolor", 2: "virginica"})
print("\nData after converting class names:")
print(df.head())

# Group by class and display statistics
print("\nStatistics:")
print(df.groupby("class").describe())

# Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    train_size=0.5,
    random_state=0
)
print("\nTraining data:", X_train.shape)
print("Testing data:", X_test.shape)

# Create Naive Bayes model
nb = GaussianNB()

# Train the model
model = nb.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)
print("\nPredicted values:")
print(y_pred)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
```

```
print("\nConfusion Matrix:")
print(cm)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:")
print(accuracy)
print("\nAccuracy Percentage:")
print(f"{accuracy * 100}%")

# Classification Report
print("\nClassification Report:")
print(
   classification_report(
      y_test,
      y_pred,
      target_names=data.target_names
   )
)

# Display confusion matrix
plt.figure(figsize=(6, 5))
plt.imshow(cm, interpolation="nearest", cmap=plt.cm.Blues)
plt.title("Confusion Matrix")
plt.colorbar()
plt.xticks([0, 1, 2], data.target_names)
plt.yticks([0, 1, 2], data.target_names)
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.show()
```

Output:

```
First five rows:
   sepal length (cm)  sepal width (cm)  ...  petal width (cm)  class
0                5.1               3.5  ...               0.2      0
1                4.9               3.0  ...               0.2      0
2                4.7               3.2  ...               0.2      0
3                4.6               3.1  ...               0.2      0
4                5.0               3.6  ...               0.2      0
[5 rows x 5 columns]

Data after converting class names:
   sepal length (cm)  sepal width (cm)  ...  petal width (cm)  class
0                5.1               3.5  ...               0.2  setosa
1                4.9               3.0  ...               0.2  setosa
2                4.7               3.2  ...               0.2  setosa
3                4.6               3.1  ...               0.2  setosa
4                5.0               3.6  ...               0.2  setosa
[5 rows x 5 columns]

Statistics:
              sepal length (cm)                                ...
                          count  mean       std  min  ...  25%  50%  75%  max
class                                                    ...
```

```
setosa      50.0  5.006  0.352490  4.3  ...  0.2  0.2  0.3  0.6
versicolor  50.0  5.936  0.516171  4.9  ...  1.2  1.3  1.5  1.8
virginica   50.0  6.588  0.635880  4.9  ...  1.8  2.0  2.3  2.5
[3 rows x 32 columns]

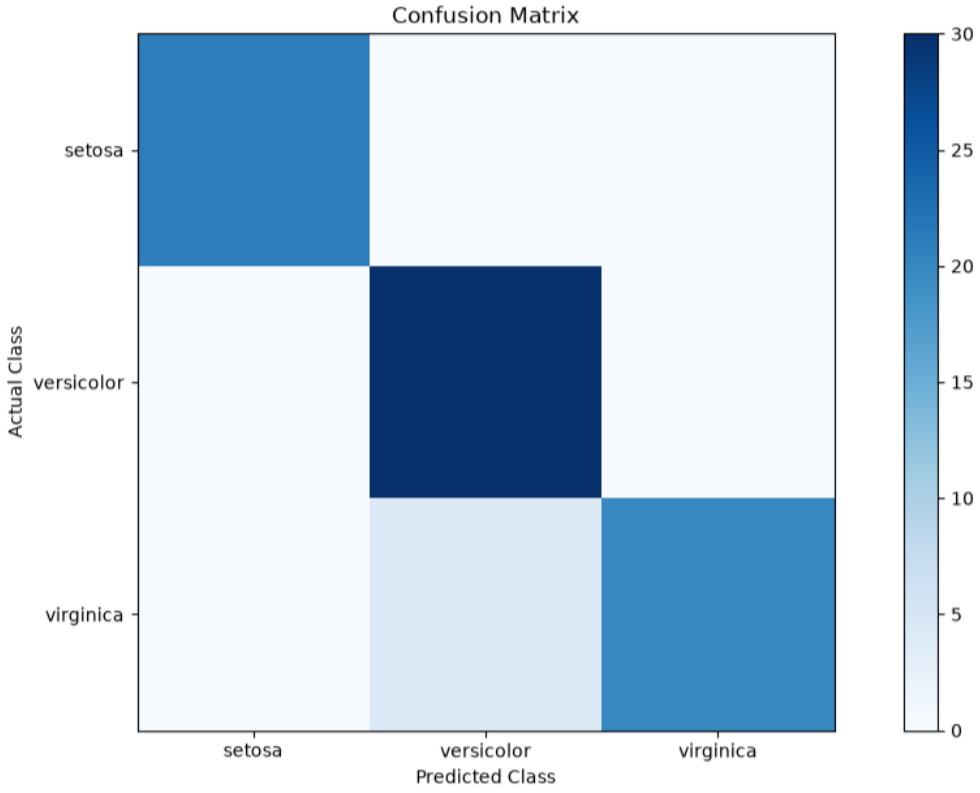
Training data: (75, 4)
Testing data: (75, 4)

Predicted values:
[2 1 0 2 0 2 0 1 1 1 1 1 1 1 1 0 1 1 0 0 2 1 0 0 2 0 0 1 1 0 2 1 0 2 2 1 0
 1 1 1 2 0 2 0 0 1 2 2 1 2 1 2 1 1 2 1 1 2 1 2 1 0 2 1 1 1 1 2 0 0 2 1 0 0 1]

Confusion Matrix:
[[21  00  00]
 [00  30  00]
 [00  04  20]]

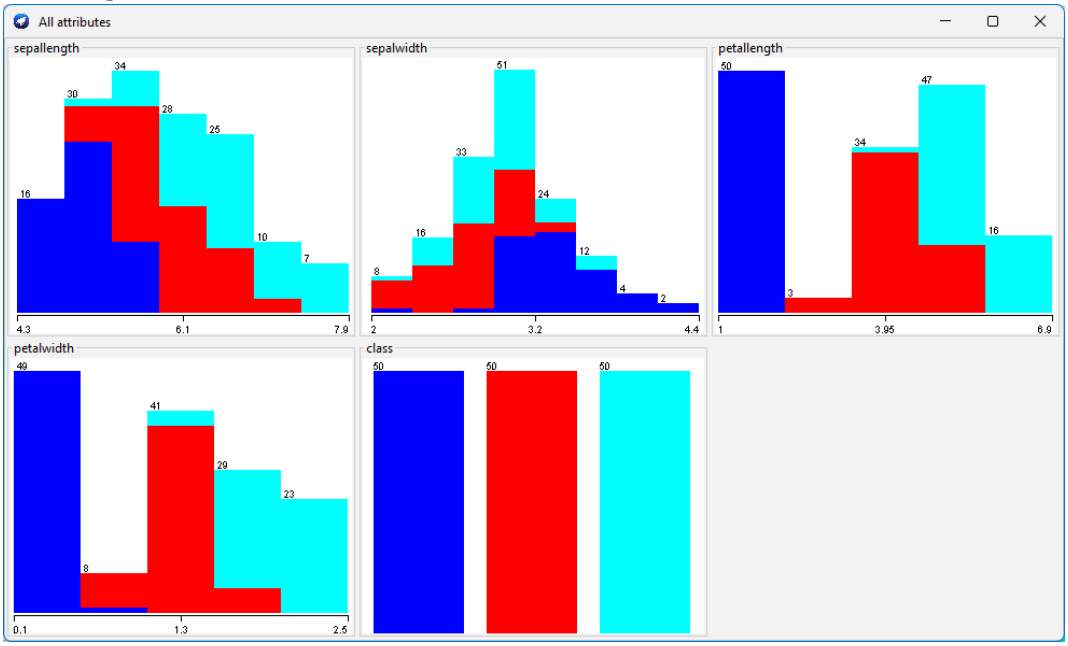
Accuracy:
0.9466666666666667
Accuracy Percentage:
94.66666666666667%

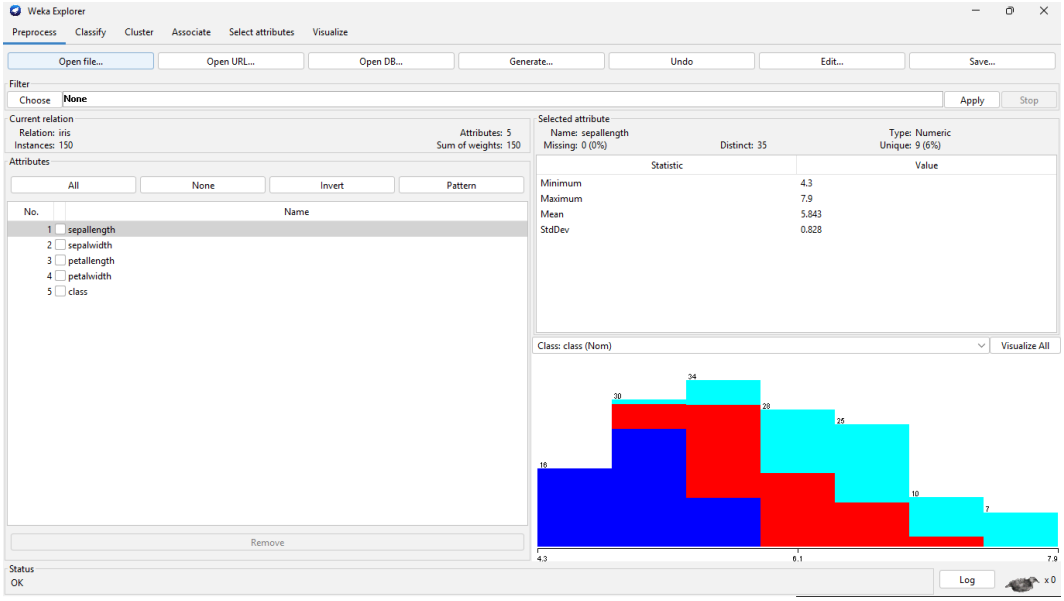
Classification Report:
              Precision Recall F1-score Support
setosa        1.00      1.00   1.00     21
versicolor    0.88      1.00   0.94     30
virginica     1.00      0.83   0.91     24
accuracy                       0.95     75
macro avg     0.96      0.94   0.95     75
weighted avg  0.95      0.95   0.95     75
```



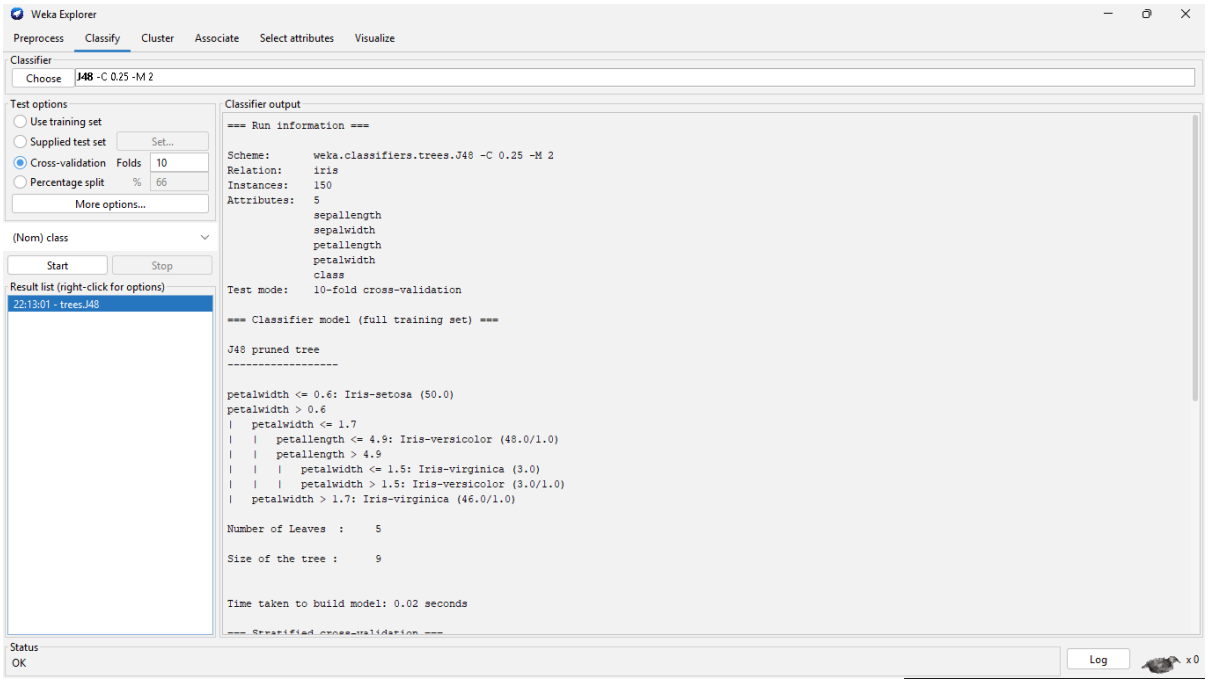
EXPERIMENT NO.06

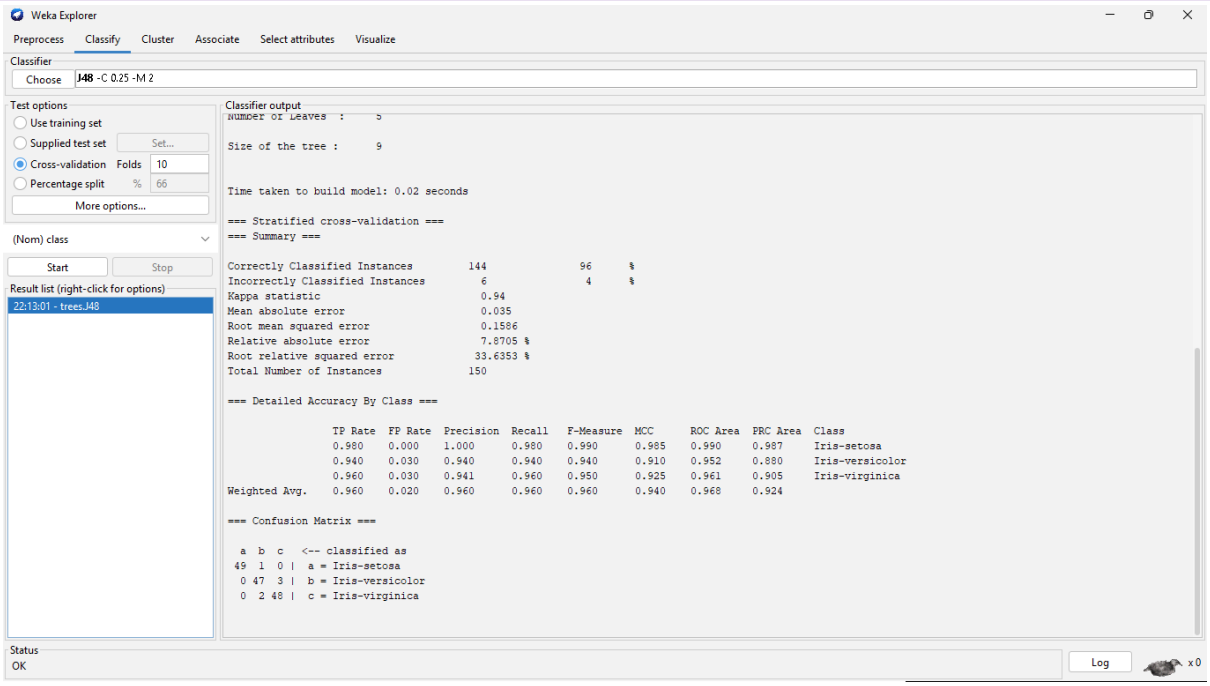
Data Preprocessing:



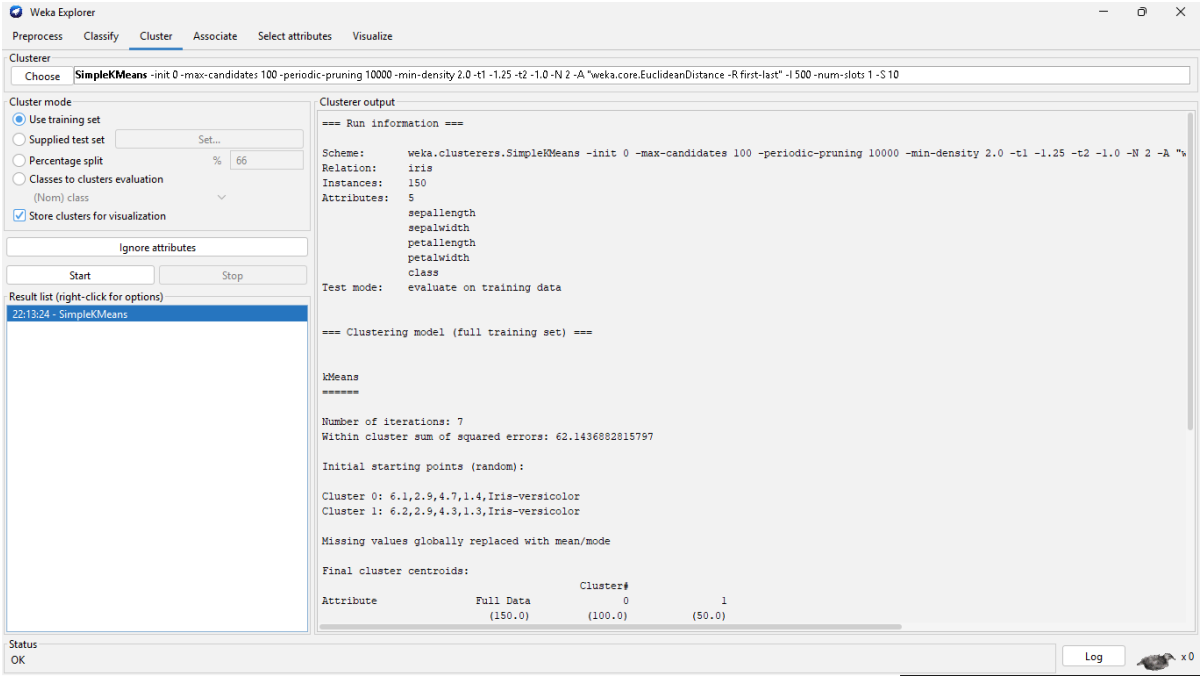


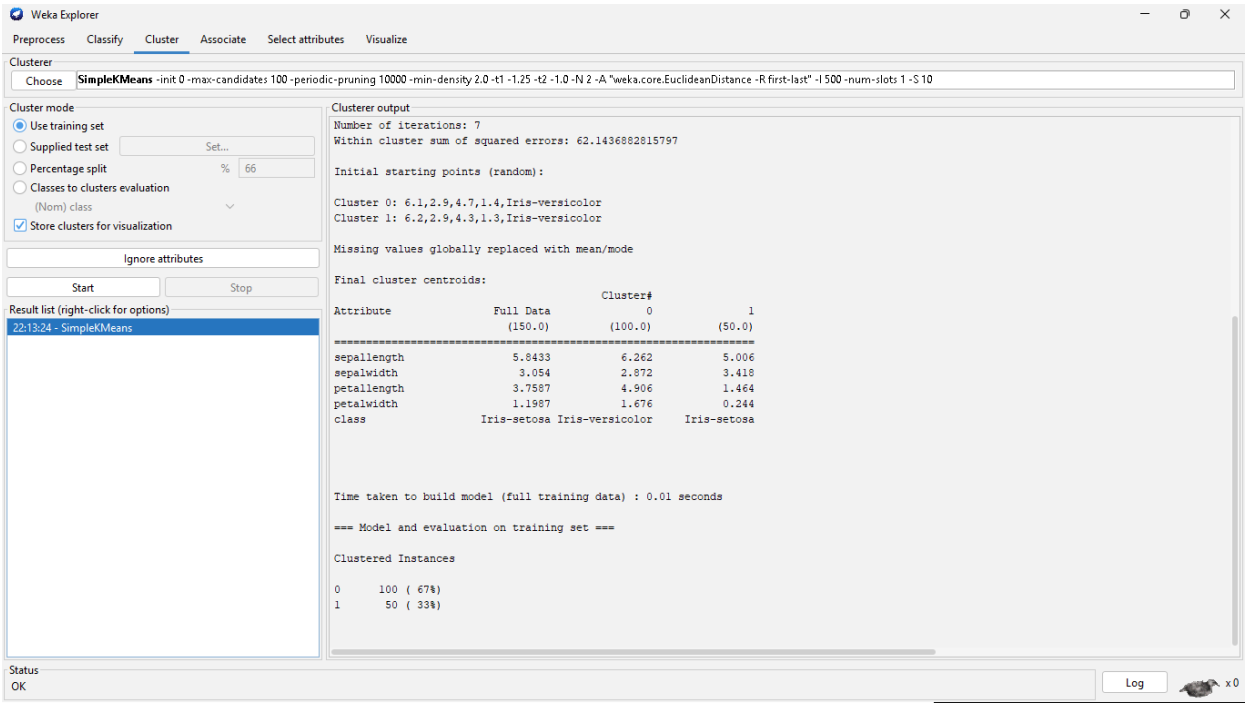
Classification:



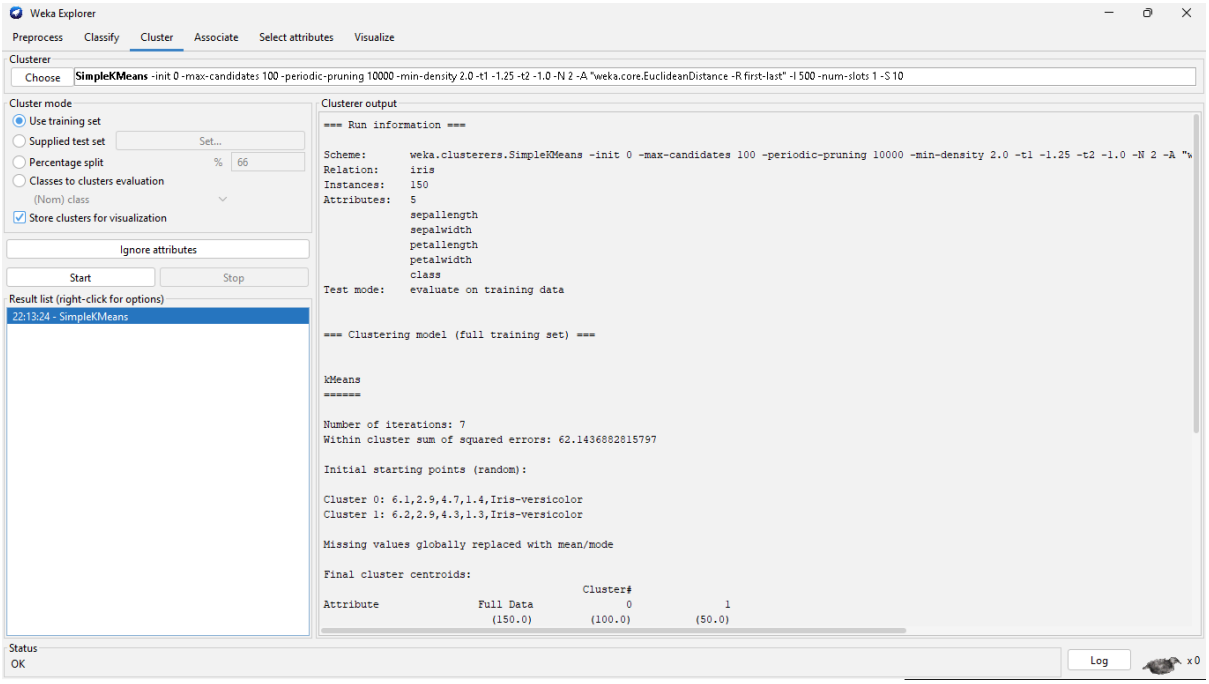

Weka Explorer
Preprocess Classify Cluster Associate Select attributes Visualize
Classifier
Choose J48 -C 0.25 -M 2
Test options
Use training set
Supplied test set Set...
Cross-validation Folds 10
Percentage split % 66
More options...
(Nom) class
Start Stop
Result list (right-click for options)
22:13:01 - trees.J48
Classifier output
Number of Leaves : 5
Size of the tree : 9
Time taken to build model: 0.02 seconds
=== Stratified cross-validation ===
=== Summary ===
Correctly Classified Instances 144 96 %
Incorrectly Classified Instances 6 4 %
Kappa statistic 0.94
Mean absolute error 0.035
Root mean squared error 0.1586
Relative absolute error 7.8705 %
Root relative squared error 33.6353 %
Total Number of Instances 150
=== Detailed Accuracy By Class ===
TP Rate FP Rate Precision Recall F-Measure MCC ROC Area PRC Area Class
0.980 0.000 1.000 0.980 0.990 0.985 0.990 0.987 Iris-setosa
0.940 0.030 0.940 0.940 0.940 0.910 0.952 0.880 Iris-versicolor
0.960 0.030 0.941 0.960 0.950 0.925 0.961 0.905 Iris-virginica
Weighted Avg. 0.960 0.020 0.960 0.960 0.960 0.940 0.968 0.924
=== Confusion Matrix ===
a b c <-- classified as
49 1 0 | a = Iris-setosa
0 47 3 | b = Iris-versicolor
0 2 48 | c = Iris-virginica
Status
OK
Log
x 0

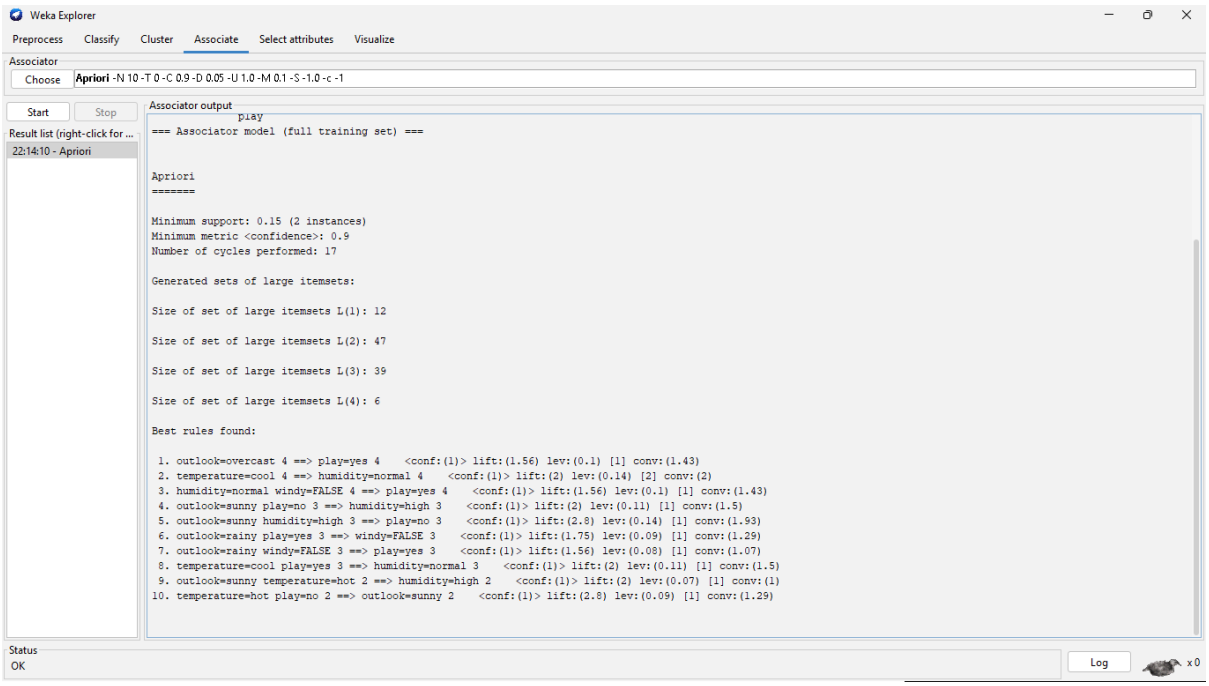
Clustering:


Weka Explorer
Preprocess Classify Cluster Associate Select attributes Visualize
Clusterer
Choose SimpleKMeans -init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 2 -A "weka.core.EuclideanDistance -R first-last" -I 500 -num-slots 1 -S 10
Cluster mode
Use training set
Supplied test set Set...
Percentage split % 66
Classes to clusters evaluation
(Nom) class
Store clusters for visualization
Ignore attributes
Start Stop
Result list (right-click for options)
22:13:24 - SimpleKMeans
Clusterer output
=== Run information ===
Scheme: weka.clusterers.SimpleKMeans -init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 2 -A "
Relation: iris
Instances: 150
Attributes: 5
sepallength
sepalwidth
petallength
petalwidth
class
Test mode: evaluate on training data
=== Clustering model (full training set) ===
kMeans
======
Number of iterations: 7
Within cluster sum of squared errors: 62.1436882815797
Initial starting points (random):
Cluster 0: 6.1,2.9,4.7,1.4,Iris-versicolor
Cluster 1: 6.2,2.9,4.3,1.3,Iris-versicolor
Missing values globally replaced with mean/mode
Final cluster centroids:
Cluster#
Attribute Full Data 0 1
(150.0) (100.0) (50.0)
Status
OK
Log
x 0

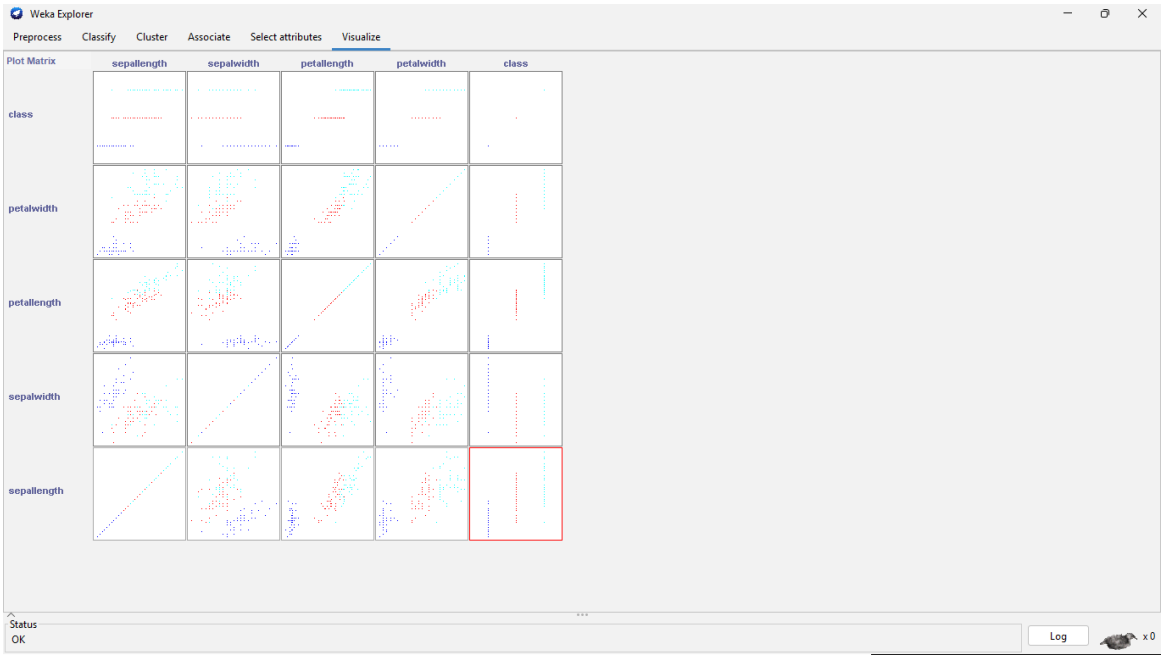

Weka Explorer
Preprocess Classify Cluster Associate Select attributes Visualize
Clusterer
Choose SimpleKMeans -init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 2 -A "weka.core.EuclideanDistance -R first-last" -I 500 -num-slots 1 -S 10
Cluster mode
Use training set
Supplied test set Set...
Percentage split % 66
Classes to clusters evaluation
(Nom) class
Store clusters for visualization
Ignore attributes
Start Stop
Result list (right-click for options)
22:13:24 - SimpleKMeans
Clusterer output
Number of iterations: 7
Within cluster sum of squared errors: 62.1436882815797
Initial starting points (random):
Cluster 0: 6.1,2.9,4.7,1.4,Iris-versicolor
Cluster 1: 6.2,2.9,4.3,1.3,Iris-versicolor
Missing values globally replaced with mean/mode
Final cluster centroids:
Cluster#
Attribute Full Data 0 1
(150.0) (100.0) (50.0)
sepallength 5.8433 6.262 5.006
sepalwidth 3.054 2.872 3.418
petallength 3.7587 4.906 1.464
petalwidth 1.1987 1.676 0.244
class Iris-setosa Iris-versicolor Iris-setosa
Time taken to build model (full training data) : 0.01 seconds
=== Model and evaluation on training set ===
Clustered Instances
0 100 (67%)
1 50 (33%)
Status
OK
Log
x 0

Association Rule:





Visualization:



EXPERIMENT NO.07

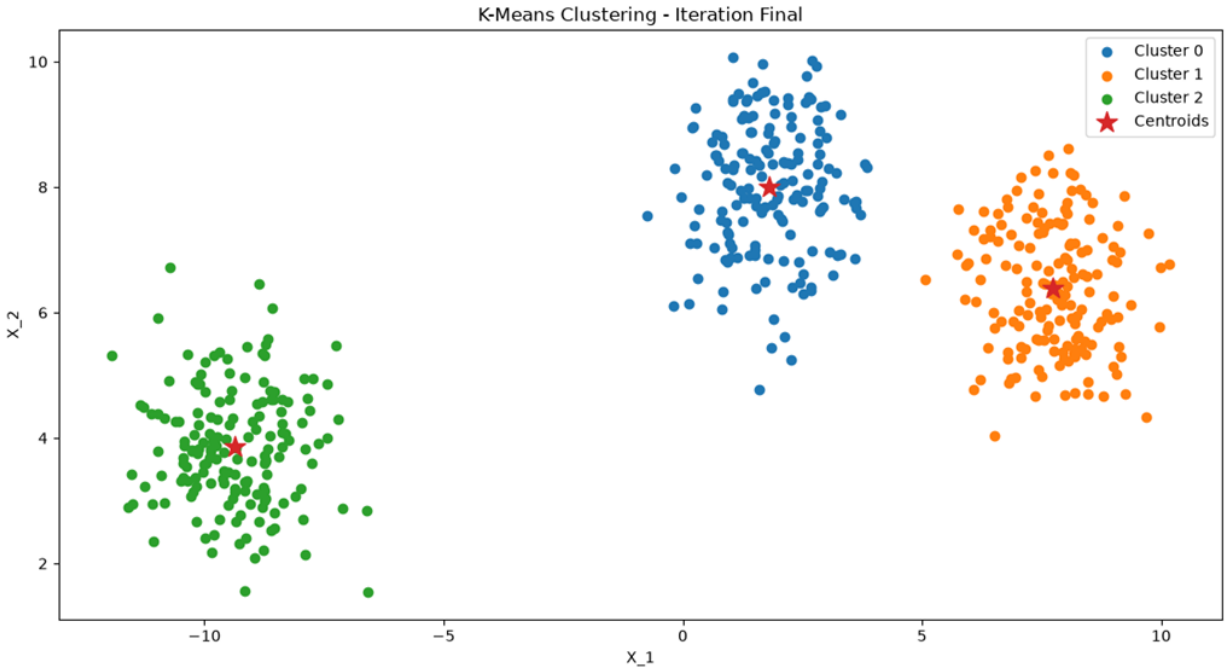
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
# 1. Initialize Centroids
def init_centroids(k, X):
    centroids = []
    for i in range(k):
        x1 = np.random.uniform(
            np.min(X[:, 0]),
            np.max(X[:, 0])
        )
        x2 = np.random.uniform(
            np.min(X[:, 1]),
            np.max(X[:, 1])
        )
        centroids.append([x1, x2])
    return np.array(centroids)
# 2. Calculate Euclidean Distance
def distance(a, b):
    return np.sqrt(np.sum((a - b) ** 2))
# 3. Assign Each Point to Nearest Cluster
def assign_cluster(k, X, centroids):
    cluster = []
    for i in range(len(X)):
        distances = []
        for j in range(k):
            d = distance(X[i], centroids[j])
            distances.append(d)
        nearest = np.argmin(distances)
        cluster.append(nearest)
    return np.array(cluster)
# 4. Calculate New Centroids
def compute_centroids(k, X, cluster):
    new_centroids = []
    for i in range(k):
        points = []
        for j in range(len(X)):
            if cluster[j] == i:
                points.append(X[j])
        new_centroids.append(np.mean(points, axis=0))
    return np.array(new_centroids)
# 5. Calculate Change in Centroids
def measure_change(old_centroids, new_centroids):
    change = 0
    for old, new in zip(old_centroids, new_centroids):
        change += distance(old, new)
    return change
# 6. Display Clusters
def show_clusters(X, cluster, centroids, iteration):
    plt.figure(figsize=(8, 6))
    # Display each cluster
    for i in range(3):
```

```
        plt.scatter(
            X[cluster == i, 0],
            X[cluster == i, 1],
            label="Cluster " + str(i)
        )
    # Display centroids
    plt.scatter(
        centroids[:, 0],
        centroids[:, 1],
        marker="*",
        s=200,
        label="Centroids"
    )
    plt.xlabel("X_1")
    plt.ylabel("X_2")
    plt.title("K-Means Clustering - Iteration " + str(iteration))
    plt.legend()
    plt.show()
# 7. K-Means Algorithm
def k_means(k, X):
    # Step 1: Initialize centroids
    centroids = init_centroids(k, X)
    change = 100
    iteration = 1
    while change > 0.001:
        # Step 2: Assign points to nearest centroid
        cluster = assign_cluster(k, X, centroids)
        # Step 3: Calculate new centroids
        new_centroids = compute_centroids( k,X,cluster)
        # Step 4: Calculate centroid movement
        change = measure_change(centroids,new_centroids)
        print("\nIteration:", iteration)
        print("New Centroids:")
        print(new_centroids)
        print("Centroid Change:", change)
        # Update centroids
        centroids = new_centroids
        iteration += 1
    return cluster, centroids
# 8. Generate Dataset
np.random.seed(20)
X_train, _ = make_blobs(
    n_samples=500,
    centers=3,
    n_features=2,
    random_state=20
)
# 9. Apply K-Means
k = 3
cluster, centroids = k_means(k, X_train)
# 10. Display Final Result
show_clusters (X_train, cluster, centroids, "Final")
```

OUTPUT:

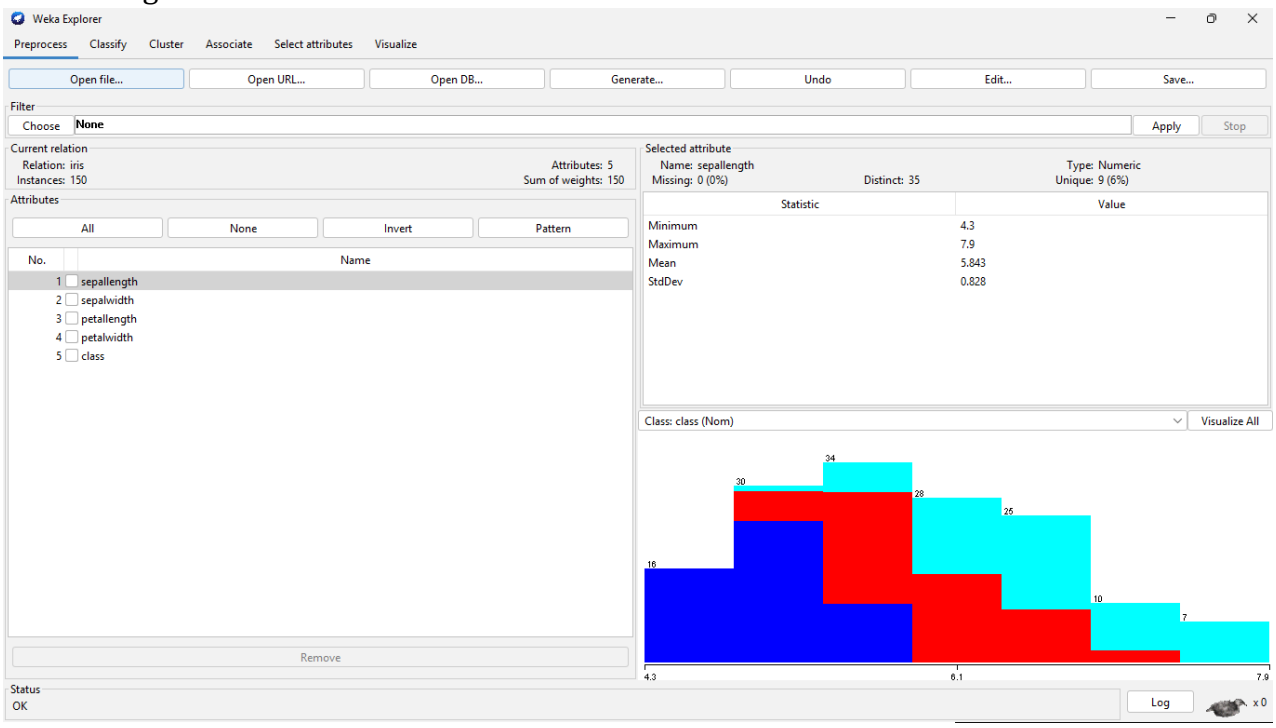
```
Iteration: 1
New Centroids:
[[ 1.80025806  7.99455026]
 [ 7.74173571  6.38572168]
 [-9.3753175   3.85672799]]
Centroid Change: 7.535172335516927

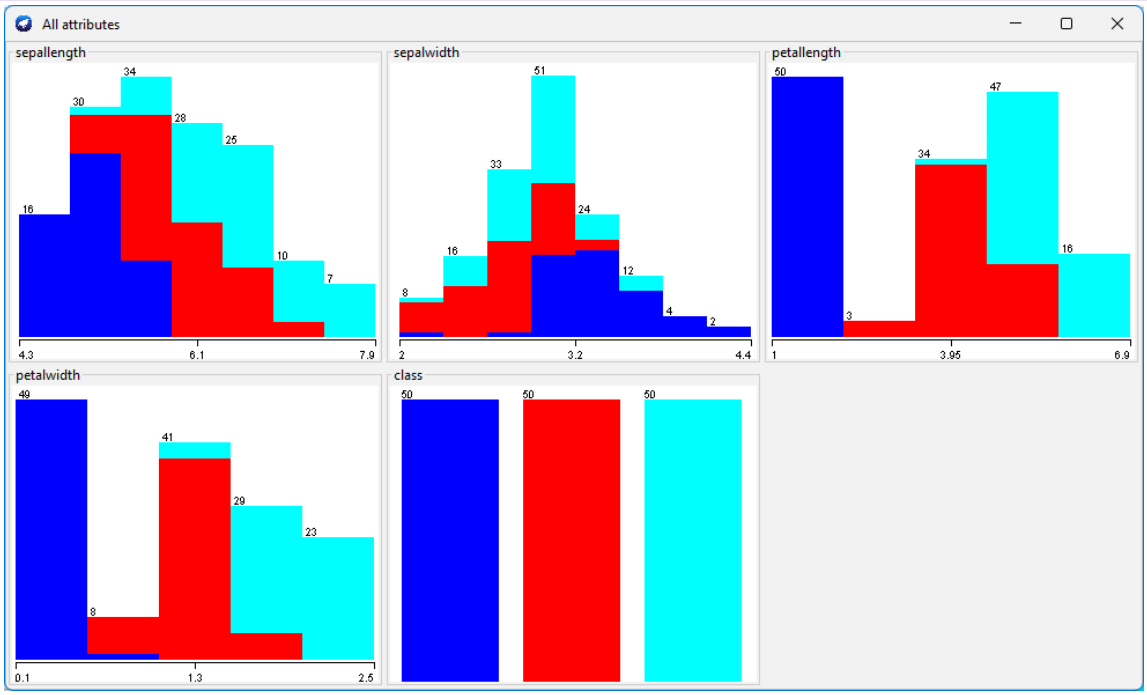
Iteration: 2
New Centroids:
[[ 1.80025806  7.99455026]
 [ 7.74173571  6.38572168]
 [-9.3753175   3.85672799]]
Centroid Change: 0.0
```



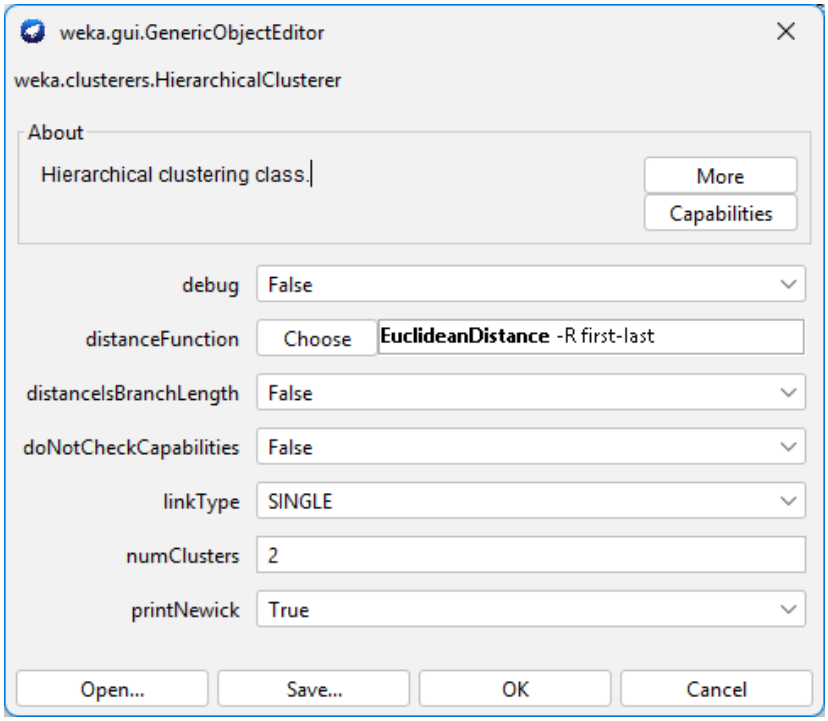
EXPERIMENT NO.07

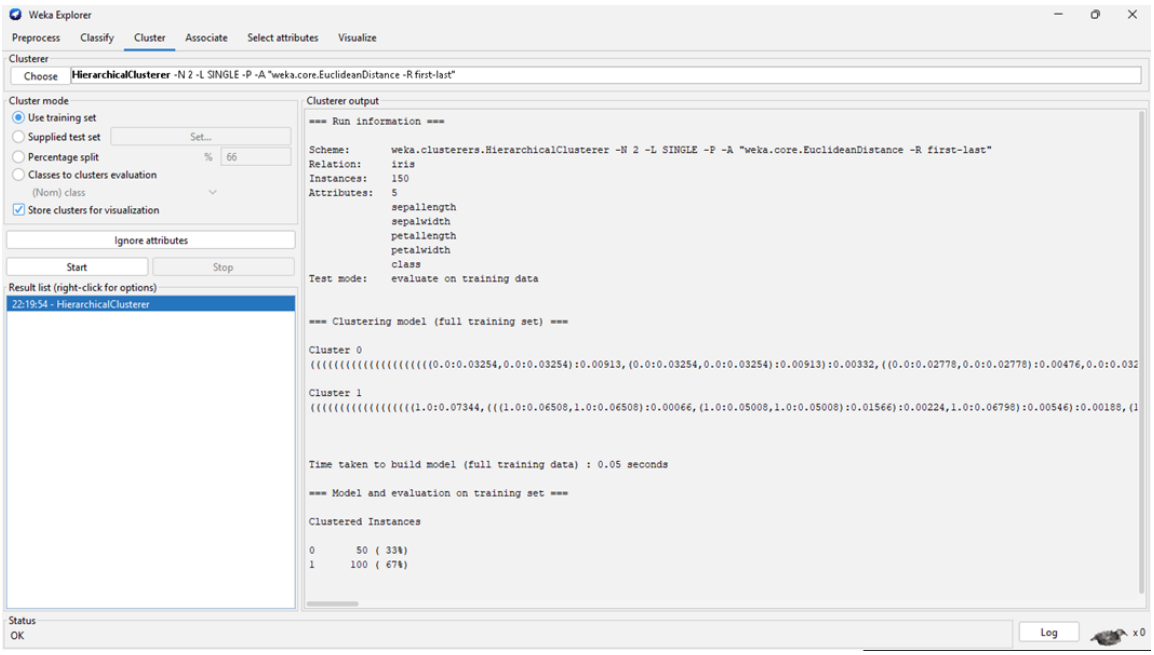
Pre Processing:

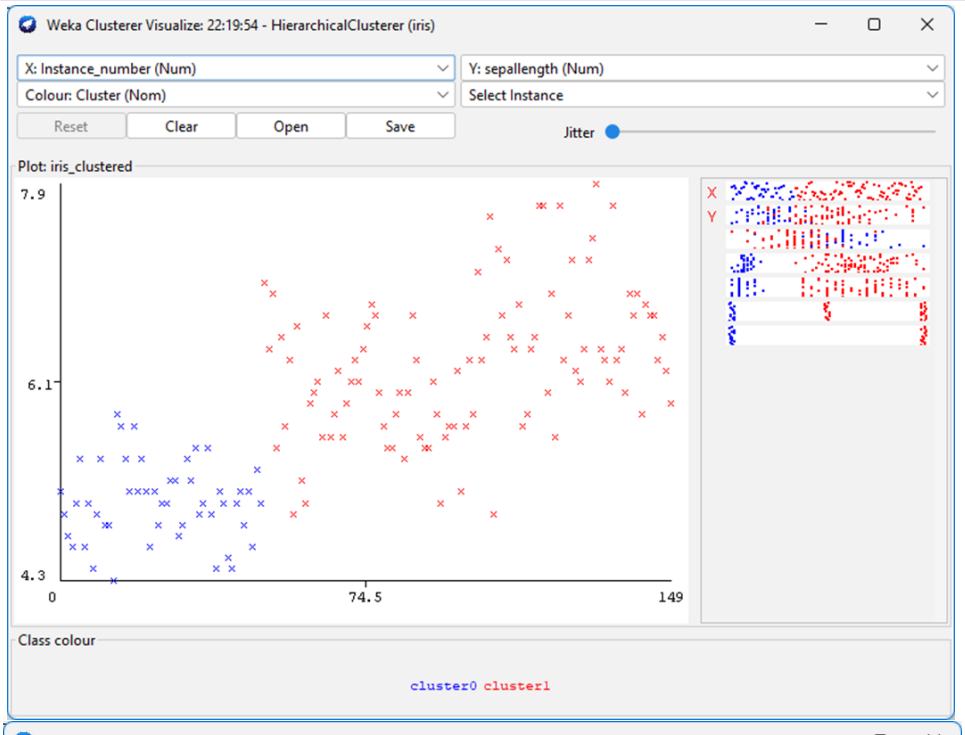


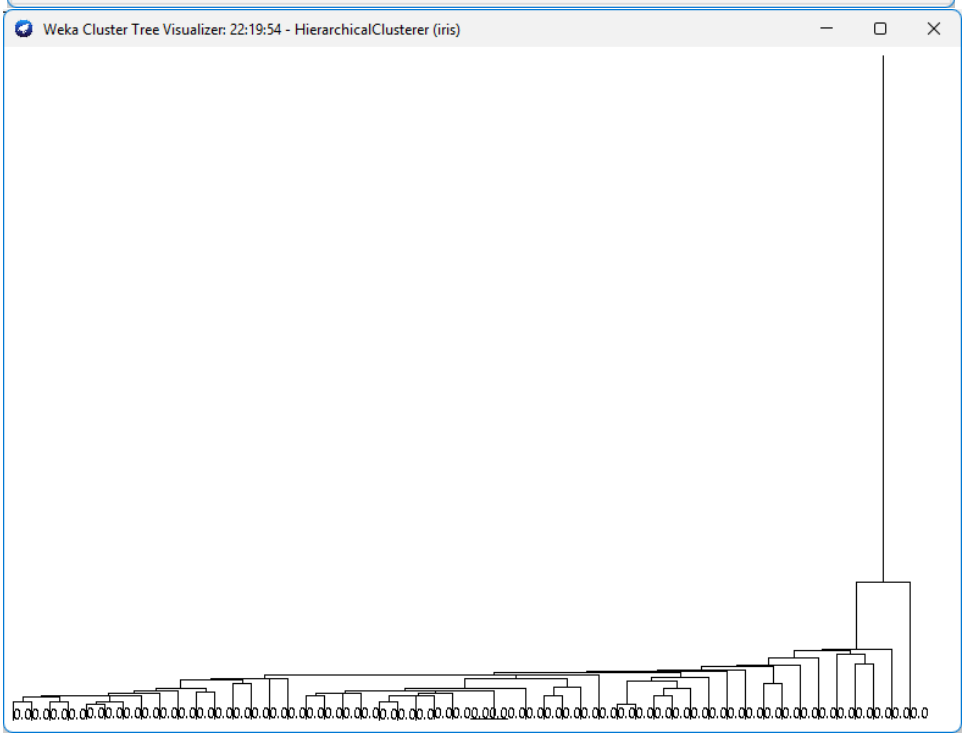


Single Linkage:

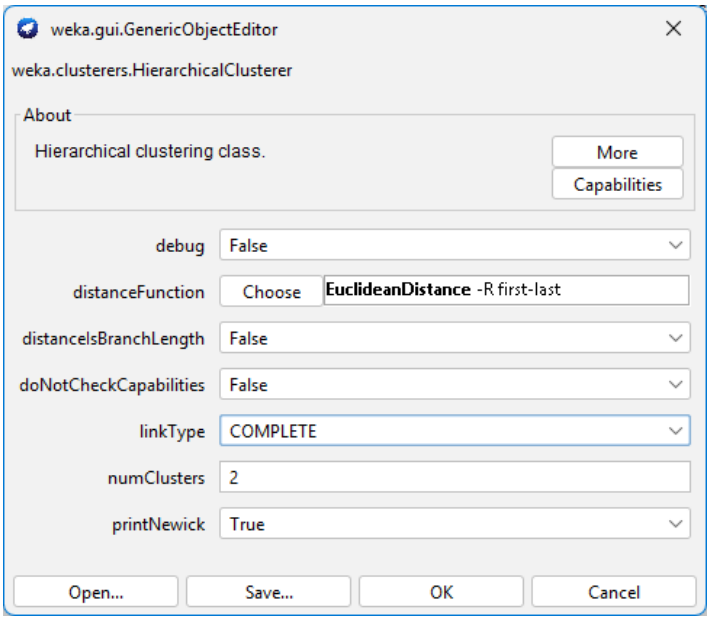


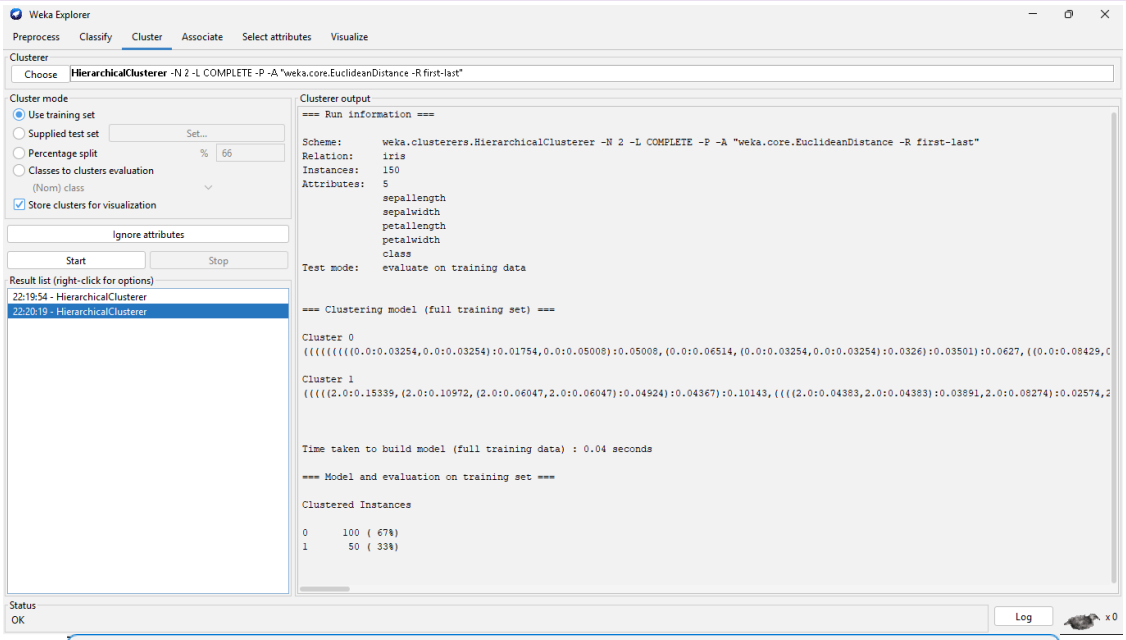


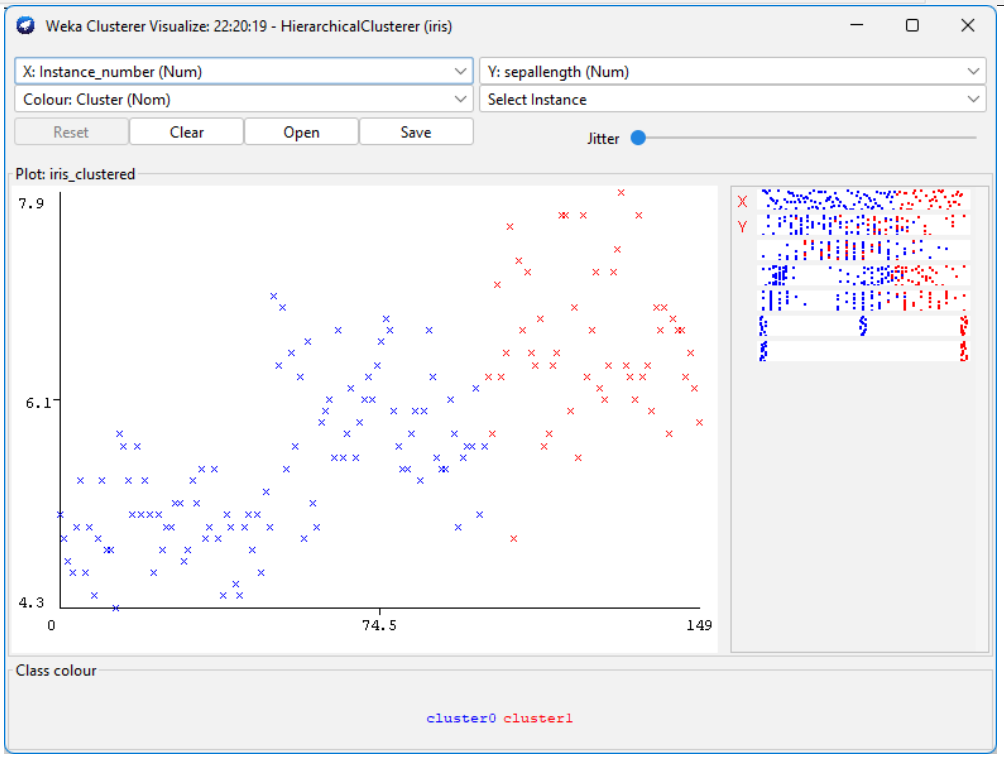


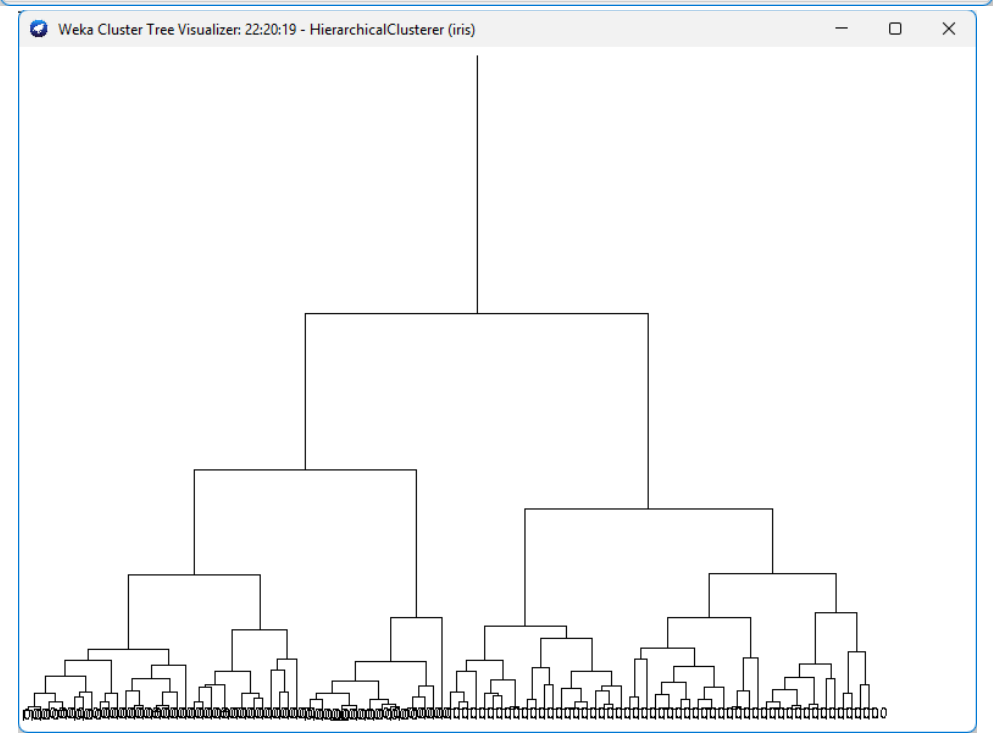


Complete Linkage:




Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Clusterer
Choose
HierarchicalClusterer -N 2 -L COMPLETE -P -A "weka.core.EuclideanDistance -R first-last"
Cluster mode
Use training set
Supplied test set
Set...
Percentage split
%
66
Classes to clusters evaluation
(Nom) class
Store clusters for visualization
Ignore attributes
Start
Stop
Result list (right-click for options)
22:19:54 - HierarchicalClusterer
22:20:19 - HierarchicalClusterer
Clusterer output
=== Run information ===
Scheme: weka.clusterers.HierarchicalClusterer -N 2 -L COMPLETE -P -A "weka.core.EuclideanDistance -R first-last"
Relation: iris
Instances: 150
Attributes: 5
sepallength
sepalwidth
petallength
petalwidth
class
Test mode: evaluate on training data
=== Clustering model (full training set) ===
Cluster 0
Cluster 1
Time taken to build model (full training data) : 0.04 seconds
=== Model and evaluation on training set ===
Clustered Instances
0 100 (67%)
1 50 (33%)
Status
OK
Log
x 0


Weka Clusterer Visualize: 22:20:19 - HierarchicalClusterer (iris)
X: Instance_number (Num)
Y: sepallength (Num)
Colour: Cluster (Nom)
Select Instance
Reset
Clear
Open
Save
Jitter
Plot: iris_clustered
7.9
6.1
4.3
0
74.5
149
X
Y
Class colour
cluster0 cluster1


Weka Cluster Tree Visualizer: 22:20:19 - HierarchicalClusterer (iris)

Average Linkage:

